# ResumeLens

## Formal-Language Models for Transparent Résumé Screening

Johan Mora • Joshua Sayur • Juan David Fajardo
Universidad Icesi | Computación y Estructuras Discretas III | 2026-2

## 01 Problem and objective

Résumés express similar qualifications with different names, spellings, and abbreviations. Literal keyword matching can miss skills or treat equivalent expressions as unrelated.

**Research objective.** Extract explicit candidate evidence, normalize technical qualifications, recognize formal patterns for four job profiles, and validate a structured candidate specification.

**Research question**

How can formal-language models make résumé screening more interpretable, consistent, and auditable?

The tool does not rank people or make hiring decisions.

## 02 End-to-end architecture

1. **Extract** Python `re`
2. **Normalize** finite-state transducers
3. **Recognize** four DFA classifiers
4. **Validate** EBNF + textX grammar
5. **Render** HTML / Markdown profile

`run_pipeline()` coordinates every stage. The same program evaluates all four supported profiles. Intermediate outputs remain inspectable.

## 03 Regex extraction

Python regular expressions identify personal/contact information, education, professional experience, and technical qualifications. The extraction stage does *not* classify a candidate.

**Example input**

```
Wednesday Addams
Technical Skills: JS, React.js, NodeJS,
Postgres, Git
```

**Extracted qualification tokens**

```
[JS, React.js, NodeJS, Postgres, Git]
```

Regex identifies textual patterns; the following FST stage resolves equivalent technology names.

## 04 Normalization with FSTs

Nine genuine `pyformlang.fst.FST` instances normalize aliases by technological family. Each is described by the formal seven-tuple:

$$T = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$$

Inputs are read character by character. Intermediate transitions emit the empty output $\varepsilon$; a completed alias produces its canonical skill token.

**Examples of transformations**

| Variant input | Canonical output |
|---|---|
| `JS` | `JAVASCRIPT` |
| `React.js` | `REACT` |
| `NodeJS` | `NODE_JS` |
| `Postgres` | `POSTGRESQL` |
| `sklearn` | `SCIKIT_LEARN` |
| `Tensor Flow` | `TENSORFLOW` |

**9 FSTs** • 35 canonical technologies

Aliases are grouped by families, rather than building one transducer per input word.

## 05 Recognition with four DFAs

Each profile is recognized by a deterministic finite automaton:

$$A = (Q, \Sigma, \delta, q_0, F)$$

States encode subsets of satisfied mandatory categories. With $k$ required categories, the shared construction creates $2^k$ states. Optional skills induce self-loops; skill order does not determine acceptance.

| Profile | States |
|---|---|
| Full Stack Developer | 32 |
| Machine Learning Engineer | 64 |
| QA Engineer | 64 |
| Data Analyst | 16 |

**Conceptual cumulative recognition**




*Schematic illustration only; the repository includes complete DOT transition diagrams for all four DFAs.*

## 06 Candidate DSL and textX

The DSL represents candidate details, education, experience, technical skills, and classification. A context-free grammar is formalized in EBNF and implemented using `resume.tx` and textX.

**Example candidate-language fragment**

```
candidate "Wednesday Addams" {
   experience { years: 3 }
   skills { JAVASCRIPT, REACT,
        NODE_JS, POSTGRESQL, GIT }
   classification {
      FULL_STACK_DEVELOPER: ACCEPTED
   }
}
```

**Two layers of validation:** lexical/syntactic grammar checks and semantic checks (nonempty names; 0–60 experience years; unique skills and unique profile results).

## 07 Results and test evidence

**345**

**automated tests passed in the final local audit**

| Sample résumé | Accepted profile |
|---|---|
| Wednesday Addams | Full Stack Developer |
| Mary Jane Watson | Machine Learning Engineer |
| QA example | QA Engineer |
| Data Analyst example | Data Analyst |

End-to-end CLI tests also covered empty input, unknown technologies, and invalid experience ranges. The latest audit did not independently verify browser interaction with the Streamlit UI.

## 08 Limitations and conclusions

**Limitations:** Results depend on explicit textual evidence; the tool cannot infer missing qualifications or assess overall hiring suitability.

**Conclusion:** Combining Regex, FSTs, DFAs and a CFG offers an interpretable, modular, testable qualification-pattern pipeline.

**Selected references:** [1] Hopcroft, Motwani & Ullman, *Introduction to Automata Theory, Languages, and Computation* (2006). [2] Mohri, *Finite-State Transducers in Language and Speech Processing* (1997). [3] textX documentation (textx.github.io).